Secure Boot on EN683

From Root of Trust to Kernel

시큐어 부트 이론과 EN683 부트 흐름

작성 2026-10-27. 근거: ~/en683_bsp SDK 소스 (U-Boot 2022.10, 빌드 스크립트, secure_boot 도구), EVK 부트 로그

Contents

1 What Secure Boot Protects 3

2 Building Blocks — 해시, 서명, 암호화, 인증서 3

3 Root of Trust & OTP — 시작점을 바꿀 수 없게 만드는 법 4

3.1 What an OTP Bit Looks Like — 퓨즈 한 비트의 실제 모습 4

4 Chain of Trust — 다음 단계를 검증한 뒤에 넘긴다 7

5 Key Hierarchy — CM 키와 DM 키를 나누는 이유 8

6 Rollback & Debug — 서명만으로는 막지 못하는 두 가지 8

7 Security Hardware — EN683 안에서 누가 검증하나 9

8 Boot Flow on EN683 — 전원부터 커널까지 11

8.1 Flash Layout — 무엇이 어디에 있나 11

8.2 EVK Boot Log — 지금 보드에서 보이는 것 12

9 Image Format & Verification — 서명된 이미지 안에는 무엇이 있나 12

9.1 Verification Steps — 검증 함수가 하는 일 12

10 Build & Sign — SDK에서 서명이 들어가는 곳 13

10.1 Switches 13

10.2 What the Makefile Signs 14

11 Provisioning — OTP에 키를 쓰고 시큐어 부트를 켜는 순서 15

12 Gaps & Checks — 체인이 끊어지는 곳 15

13 Troubleshooting — 실패 메시지 읽기 16

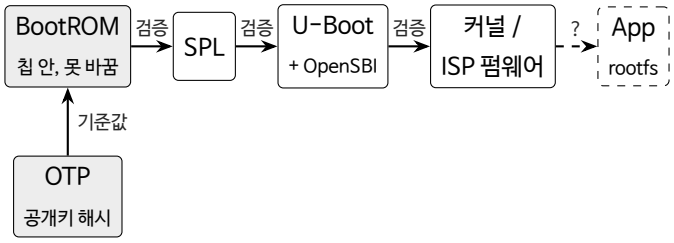
13.1 Fail Messages — 번호별 원인 16

13.2 sboot — 보드에서 검증만 해 보기 17

14 Glossary & References **17**

Part I. Theory

1. What Secure Boot Protects



개념도. 각 단계는 다음 단계를 실행하기 전에 서명을 검증한다. 점선은 EN683 SDK 기본 상태에서 검증이 이어지지 않는 구간(12장).

시큐어 부트의 목적은 하나다. **제조사가 서명한 코드만 실행되게 하는 것**이다. 누군가 플래시를 떼어 내 다른 펌웨어를 써 넣거나, 업데이트 경로로 변조된 이미지를 밀어 넣어도, 서명이 맞지 않으면 그 단계에서 부팅이 멈춘다.

생각해야 할 것은 이 체인이 **어디서 시작하고 어디서 끝나는지**다.

- **시작점**은 바꿀 수 없어야 한다. 칩 안의 BootROM과, 한 번 쓰면 지울 수 없는 OTP가 그 역할을 한다. 시작점을 바꿀 수 있다면 그 뒤의 검증은 모두 의미가 없다.
- **끝점**을 알아야 한다. 체인이 커널에서 끝나면 rootfs와 App은 보호받지 못한다. 공격자는 체인이 끝난 다음 단계를 노린다.

시큐어 부트가 막지 못하는 것도 분명히 알아 두어야 한다. 부팅이 끝난 뒤 정상 서명된 코드의 취약점을 이용한 공격(원격 코드 실행, 권한 상승)은 막지 못한다. 시큐어 부트는 **시작 시점의 무결성**만 보장한다. 런타임 보호는 시큐어 코딩, 권한 분리, 업데이트 서명 검증이 따로 맡아야 한다.

2. Building Blocks — 해시, 서명, 암호화, 인증서

도구	무엇을 보장하나	무엇을 못 하나	EN683에서
CRC, MD5	우발적 손상 감지	공격자는 내용을 바꾸고 값을 다시 계산하면 된다	지금 EVK 부트 로그의 `crc32+ md5+ OK`
SHA-256 해시	내용이 1비트라도 바뀌면 값이 달라짐	누가 만들었는지는 모른다	OTP의 공개키 해시 (SBPKD), 서명 대상
ECDSA P-256 서명	개인키를 가진 쪽이 만들었다는 것(인증)	내용을 숨기지 못한다	모든 부트 이미지 (BLp, BLw)
AES-256 암호화	내용을 숨김(기밀성)	단독으로는 변조를 막지 못한다	BLw 모드 이미지
인증서	상위 키가 하위 공개키를 보증	상위 키가 유출되면 전체가 무너진다	CM 키가 DM 공개키에 서명 (DM.cert.bin)

가장 흔한 오해가 **해시 검증을 보안 검증으로 착각하는 것**이다. EVK 부트 로그에서 U-Boot가 커널 FIT 이미지를 읽을 때 `Verifying Hash Integrity ... crc32+ md5+ OK`가 나온다. 이것은 플래시에서 읽은 데이터가 깨지지 않았는지만 본다. 공격자가 커널을 바꾸고 FIT 안의 해시값도 새로 계산해 넣으면 그대로 통과한다. 누가 만들었는지를 확인하려면 **개인키로 만든 서명**이 있어야 하고, 검증하는 쪽은 **미리 믿기로 정한 공개키**를 갖고 있어야 한다.
서명 검증은 두 단계로 이루어진다. 먼저 이미지의 SHA-256 해시를 계산하고, 다음에 서명이 그 해시를 해당 개인키로 서명한 것인지 공개키로 확인한다. 그래서 서명 알고리즘(ECDSA P-256)과 해시 알고리즘(SHA-256)은 항상 짝으로 나온다.
암호화는 목적이 다르다. 펌웨어를 플래시에서 읽어 내 역공학하는 것을 어렵게 한다. EN683의 BLw 모드는 **서명과 암호화를 함께** 적용한다. 서명이 없는 암호화는 변조를 막지 못하므로, 암호화만 켜고 안심하면 안 된다.

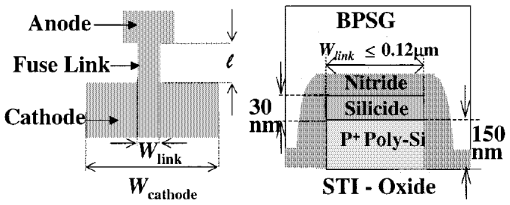
3. Root of Trust & OTP — 시작점을 바꿀 수 없게 만드는 법

체인의 첫 검증자는 **BootROM**이다. 칩을 만들 때 마스크로 새겨지므로 소프트웨어로 바꿀 수 없다. BootROM이 쓸 기준 공개키는 칩마다 다른 제품 회사의 키여야 하므로 ROM에 넣을 수 없다. 그래서 **OTP**(One-Time Programmable, 한 번만 쓸 수 있는 퓨즈 메모리)에 둔다.

OTP에는 공개키 자체가 아니라 **공개키의 SHA-256 해시**(32바이트)를 넣는다. OTP는 비싸고 작기 때문이다. EN683 SDK의 CM 공개키는 64바이트(P-256의 X, Y 좌표)이고, 해시는 32바이트다. 검증 때는 이미지 헤더에 들어 있는 공개키를 꺼내 해시를 계산하고, OTP의 해시와 같은지 비교한다. 같으면 그 공개키를 믿고, 그 키로 서명을 검증한다. SDK에서 이 값이 `en683_sb_SBPKD.bin`(Secure Boot Public Key Digest)이고 **RoT-OTP 자산 번호 4**에 쓴다.
OTP는 되돌릴 수 없다. 잘못된 해시를 쓰거나, 서명된 이미지를 플래시에 넣기 전에 시큐어 부트 활성 플래그를 켜면 그 칩은 다시 부팅하지 못한다. 그래서 OTP 쓰기는 개발 보드가 아니라 **양산 공정의 정해진 단계**에서 한다(11장). 현재 EVK가 시큐어 부트를 켜지 않은 이유도 이것이다.

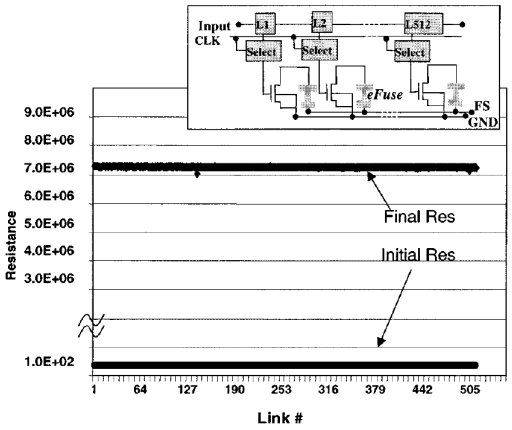
3.1 What an OTP Bit Looks Like — 퓨즈 한 비트의 실제 모습

OTP 한 비트는 칩 안의 아주 작은 퓨즈다. 방식은 크게 두 가지다. **eFuse**는 처음에 연결된 금속·실리사이드 선을 큰 전류로 끊어 저항을 높이고, **안티퓨즈**(antifuse)는 처음에 끊겨 있는 얇은 게이트 산화막을 높은 전압으로 깨뜨려 연결한다. 어느 쪽이든 물리 구조를 바꾸는 것이므로 되돌릴 수 없다.



eFuse 구조: 위에서 본 모양(왼쪽)과 단면(오른쪽)

출처: C. Kothandaraman, S. K. Iyer, S. S. Iyer, "Electrically Programmable Fuse (eFUSE) Using Electromigration in Silicides," IEEE EDL, vol. 23, no. 9, 2002, Fig. 1

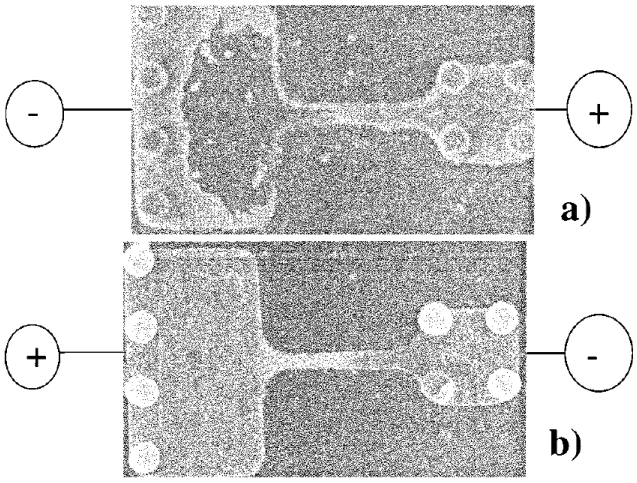


퓨즈 512개의 프로그램 전(아래 줄)과 후(위 줄) 저항

출처: 같은 논문, Fig. 2

왼쪽 그림에서 폭이 0.12μm 이하인 가는 선(fuse link)이 큰 양극(anode)과 음극(cathode) 사이에 있다. 단면을 보면 P^+ 폴리실리콘 위에 실리사이드가 얹혀 있고, 그 위를 질화막과 BPSG가 덮는다. 일반 CMOS 공정의 게이트 층을 그대로 쓰므로 공정을 추가하지 않아도 된다.

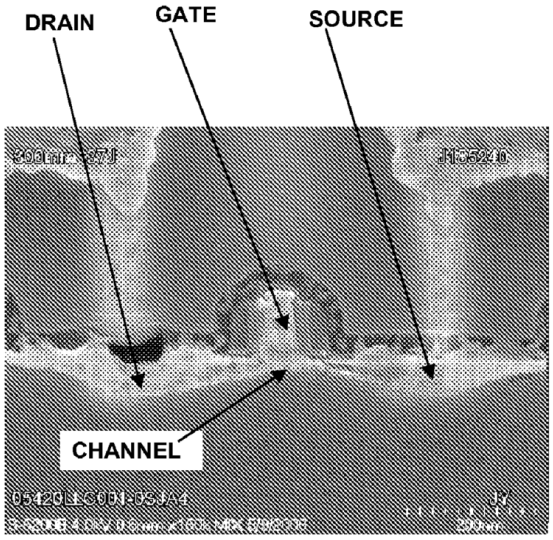
오른쪽 그림은 512개 퓨즈의 저항이다. 프로그램 전에는 모두 약 100Ω대(논문 본문 기준 약 120Ω)에 있고, 1.5V, 200μs 펄스로 프로그램한 뒤에는 약 7MΩ(본문 기준 $7 \times 10^6\Omega$ 이상)로 올라갔다. 두 값이 수만 배 떨어져 있어서 읽기 회로가 0과 1을 쉽게 구분한다. 그래프 위의 회로는 래치(L1~L512)가 퓨즈 하나씩을 골라 FS 전원으로 전류를 흘리는 구조다. 이 FS가 쓰기 전용 전원이다.



프로그램한 eFuse를 위에서 본 SEM 사진. (a)와 (b)는 전류 방향을 바꾼 것

출처: Kothandaraman et al., IEEE EDL 2002, Fig. 3

사진에서 어두운 부분은 실리사이드가 빠져나간 영역이고, 밝은 부분은 실리사이드가 쌓인 영역이다. 전류를 흘리면 전자에 밀려 실리사이드의 금속(코발트)이 양극 쪽으로 이동한다(electromigration). 음극 근처의 선이 비면서 저항이 크게 오른다. (a)와 (b)에서 전류 방향을 바꾸면 빈 영역도 반대쪽으로 옮겨 가는데, 이것이 열로 녹아 끊어진 것이 아니라 전자 이동으로 프로그램되었다는 증거다. 논문은 음극을 크게 만들어 온도 기울기를 키우면 프로그램이 더 잘 된다고 설명한다.



프로그램한 뒤의 MOSFET 퓨즈 단면 SEM

출처: S. N. Chakravarti et al., US Patent 7,531,886 B2 (IBM), 2009, Fig. 3a

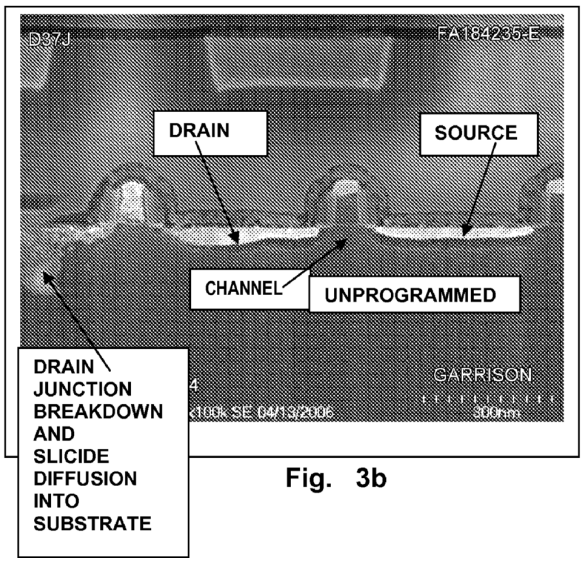


Fig. 3b

같은 소자의 프로그램 전 상태 SEM

출처: US 7,531,886 B2, Fig. 3b

이 특허는 트랜지스터 자체를 퓨즈로 쓴다. 얇은 게이트 산화막 NFET의 드레인에 약 5V, 10μs 펄스를 주면, 드레인 접합이 깨지고 실리사이드가 채널 쪽으로 이동해 채널이 전도성이 된다(왼쪽). 프로그램 전(오른쪽)에는 채널에 실리사이드가 없다. 특허는 프로그램 후 소스 전류가 프로그램 전의 10^4배 이상이라고 적는다. 오른쪽 사진 왼쪽 아래의 "DRAIN JUNCTION BREAKDOWN" 표시는 특허 원본의 배치 그대로이고, 특허 본문은 이 현상을 3a(프로그램 후)에서 보인다고 설명한다.

세 자료를 EN683과 이어 보면 두 가지를 생각해야 한다. 첫째, **OTP 쓰기에는 평소보다 큰 전압과 전류가 필요하다**. EN683 U-Boot가 쓰기 전에 `OTP_IOCTL_PWEN`으로 별도 전원을 켜는 이유가 이것이다. 이 전원이 처지면 일부 비트가 덜 프로그램되어 저항이 애매한 값에 머물 수 있으므로, 쓰기 후 검증(`rot_OTPScan`)이 반드시 필요하다. 둘째, **OTP 값은 물리적으로 관찰할 수 있다**. 칩을 깎아 SEM으로 보면 어느 퓨즈가 끊겼는지 읽을 수 있다. 그래서 OTP에는 공개되어도 괜찮은 공개키 해시를 넣고, 비밀 키(HUK, SBCR)는 보안 모듈 안에서만 쓰고 바깥으로 꺼내지 않는 구조를 쓴다. EN683의 OTP가 eFuse인지 안티퓨즈인지는 SDK에서 확인하지 못했다.

EN683의 OTP 관련 자산은 SDK의 `conf/prov/metadata.txt`와 `keygenAllAuto.sh`에서 확인할 수 있다.

자산	파일	역할
3 SBCR	`en683_sb_SBCR.bin` (32B)	Secure Boot Confidentiality Root. BLw 이미지의 AES 키를 풀 때 쓰는 키 암호화 키
4 SBPKD	`en683_sb_SBPKD.bin` (32B)	CM 공개키의 SHA-256 해시. 서명 검증의 기준점
0x61 HUK	(기본 비활성)	Hardware Unique Key. 칩마다 다른 256비트 키. 칩 밖으로 꺼낼 수 없다
0x62 SFWBPKD, 0x63 SFWBCR	(기본 비활성)	보안 모듈 펌웨어 (EIP-130) 자체의 서명·암호화 기준
0x64 SFWBVER	(기본 비활성)	보안 모듈 펌웨어 버전. 롤백 방지에 쓰이는 것으로 보인다
CPU-OTP 0x7FE	`otpw 0x7fe 0xffff`	시큐어 부트 활성 플래그. Provisioning의 마지막에 쓴다

metadata.txt에서 3, 4만 활성이고 0x61~0x64는 주석 처리되어 있다. SFWBVER의 정확한 동작은 벤더 문서로 확인해야 한다.

4. Chain of Trust — 다음 단계를 검증한 뒤에 넘긴다

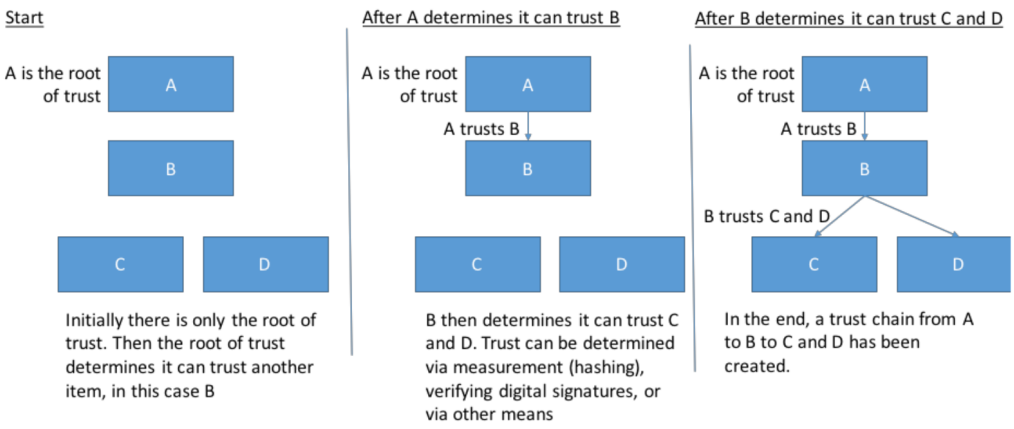
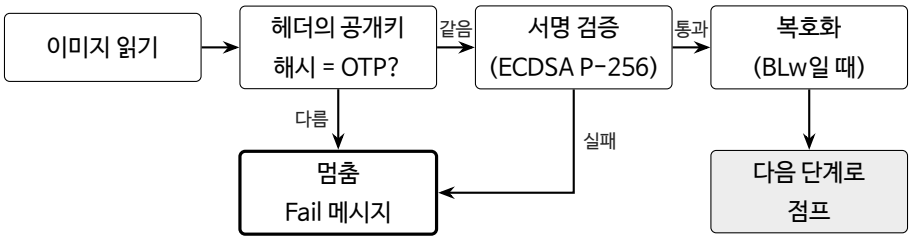


Figure 2: Roots of Trust

NIST가 설명하는 신뢰 체인: 신뢰의 출발점 A가 B를 검증하고, B가 C와 D를 검증한다

출처: NIST SP 800-193, "Platform Firmware Resiliency Guidelines," 2018, Figure 2: Roots of Trust

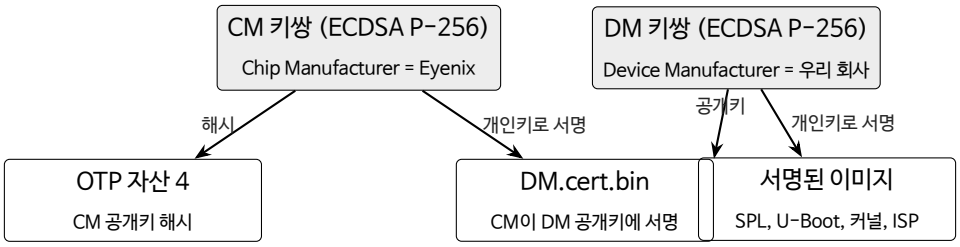
NIST 그림은 세 단계로 읽는다. 처음에는 A(신뢰의 출발점)만 믿을 수 있다. A가 B를 해시나 서명으로 확인하면 B도 믿을 수 있게 되고, 다시 B가 C와 D를 확인해 A→B→C, D로 이어지는 신뢰 체인이 만들어진다. EN683에 대응하면 A가 BootROM과 OTP, B가 SPL, C가 U-Boot, D가 ISP 펌웨어나 커널이다. 이 그림에서 생각해야 할 것은 **B가 C를 검증하는 능력도 A에게서 받은 것**이라는 점이다. B가 검증 없이 실행되면 C와 D에 대한 검증도 믿을 수 없다.



개념도. EN683 검증 라이브러리(liben683sb.a)는 소스가 없는 바이너리라서, 실제 내부 순서는 도구 인자와 함수 이름에서 추론했다(9장).

체인의 규칙은 단순하다. **각 단계는 다음 단계를 메모리에 올리고, 서명을 검증하고, 통과했을 때만 점프한다.** 실행한 뒤에 검증하면 이미 늦다. 변조된 코드가 먼저 실행되어 검증 코드 자체를 건너뛸 수 있기 때문이다.
이 방식을 **verified boot**라고 한다. 비슷한 말로 measured boot가 있다. measured boot는 실패해도 멈추지 않고, 각 단계의 해시를 TPM 같은 곳에 기록해 두었다가 나중에 원격 서버가 판단하게 한다. EN683 SDK가 구현한 것은 verified boot다. 검증에 실패하면 `Secure-Boot Fail case N`을 출력하고 그 단계에서 멈춘다.
검증 코드도 체인 안에 있어야 한다. SPL이 U-Boot를 검증하려면 SPL 자신이 먼저 BootROM에게 검증받아야 하고, 그래야 SPL 안의 검증 코드와 기준 정보를 믿을 수 있다.

5. Key Hierarchy — CM 키와 DM 키를 나누는 이유



`util/secure_boot/keygen/keygenCM.sh`, `keygenDM.sh`, `certgenDM.sh`의 동작을 그렸다.

SDK는 키를 두 단계로 나눈다. **CM 키**는 칩 회사(Eyenix) 쪽 키이고, 그 공개키 해시가 OTP에 들어간다. **DM 키**는 제품 회사가 만드는 키이고, 실제 이미지 서명에 쓴다. CM 개인키로 DM 공개키에 서명한 것이 `DM.cert.bin`이다.
검증하는 쪽은 이렇게 믿음을 넘긴다. OTP의 해시로 CM 공개키를 믿고, CM 공개키로 DM 인증서를 믿고, DM 공개키로 이미지를 믿는다. 이렇게 나누면 OTP를 다시 쓰지 않고도 DM 키를 바꿀 수 있다. CM 키로 새 DM 인증서만 만들면 되기 때문이다. 대신 **CM 개인키가 유출되면 OTP를 쓴 모든 칩이 위험해지므로** CM 키는 가장 엄격하게 보관해야 한다.
SDK의 `keyset`에 들어 있는 키는 **데모 키**이고 개인키가 공개되어 있다. WARNING.txt도 양산 전에 반드시 지우고 `keygenAllAuto.sh`로 새로 만들라고 적고 있다. `CM.der`, `DM.der`, `DM.cert.bin`은 어떤 경우에도 외부로 나가면 안 된다.

6. Rollback & Debug — 서명만으로는 막지 못하는 두 가지

롤백 공격. 예전에 정식 서명된, 그러나 취약점이 있는 펌웨어를 다시 써 넣으면 서명 검증은 통과한다. 이것을 막으려면 OTP에 **최소 허용 버전**을 두고, 이미지의 버전이 그보다 낮으면 거부해야 한다. EN683 라이브러리에는 이미지 속성을 검사하는 함수(`SBIF_Attribute_Check`)가 있지만, SDK 기본 설정에서 부트 이미지의 버전 번호를 OTP와 비교하는지는 확인하지 못했다. 보안 모듈 펌웨어용 SFWBVER 자산은 기본 설정에서 주석 처리되어 있다.

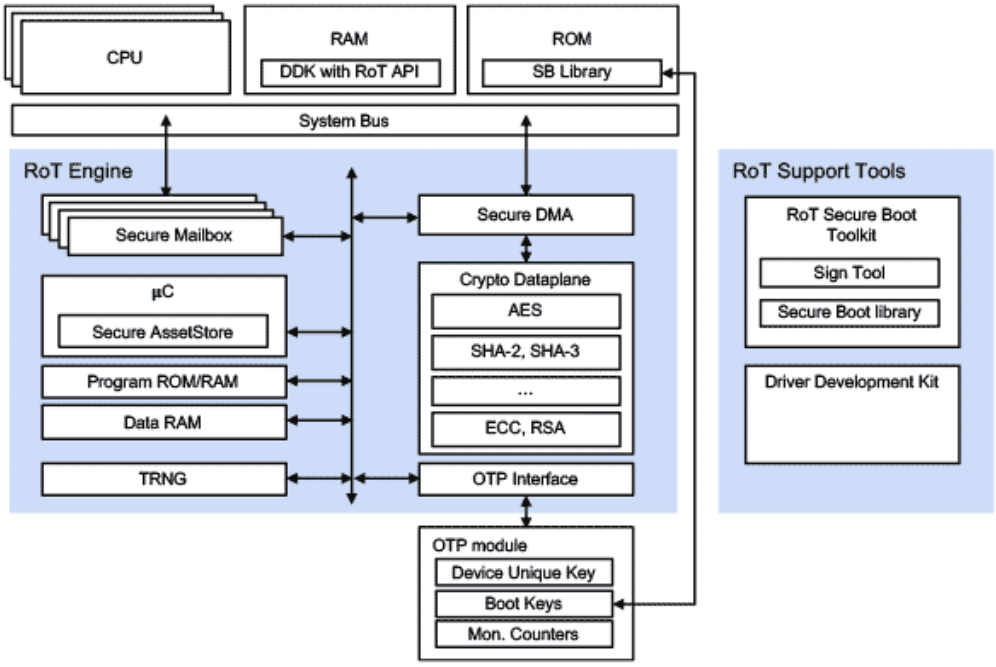
디버그 포트. JTAG가 열려 있으면 부팅이 끝난 뒤 메모리와 레지스터를 직접 읽고 쓸 수 있으므로, 시큐어 부트의 의미가 줄어든다. EN683 보안 모듈 설정(`util/gen_rotfw/config/customization.cfg`)에는 **Secure JTAG 인증 공개키** 항목이 있고, 기본값은 Eyenix 기본 키다. 양산 제품은 이 키를 자기 키로 바꾸거나 디버그 포트를 잠가야 한다. U-Boot 콘솔도 같은 문제다. 시리얼로 U-Boot 명령을 칠 수 있으면 검증되지 않은 커널을 메모리에 올려 `bootm`으로 실행할 수 있으므로, 양산 빌드에서는 콘솔 인증이나 비활성화를 함께 생각해야 한다.

Part II. EN683

7. Security Hardware — EN683 안에서 누가 검증하나

구성 요소	SDK에서 확인한 것
CPU	RISC-V 64비트(rv64imafdc), 여러 hart. hart 3은 비디오코어(ISP) 펌웨어를 실행한다(`HART3BOOT_FOR_VIDEOCORE`)
BootROM	`info.inb`(info.ini를 변환한 부트 설정)를 읽고 [boot0]~[boot5] 항목을 차례로 처리한다
보안 모듈 EIP-130	Rambus 계열 하드웨어 보안 모듈. 펌웨어 `firmware_eip130ram.sbi`를 [boot0]으로 먼저 올린다. AES, ECDSA, 키 언랩, OTP 접근을 맡는다
RoT-OTP	보안 모듈만 접근하는 OTP. SBCR, SBPKD, HUK 등 자산 저장
CPU-OTP	`otp@42b00000`, 2048바이트. 온도 센서 보정값(0x7F0)과 시큐어 부트 활성 플래그(0x7FE)가 있다
검증 라이브러리	`conf/liben683sb.a`. SPL과 U-Boot에 링크된다. `en683sb_decrypt_and_verify_auto_image()`

출처: `conf/device_tree_en683.dts`, `script/create_ini.sh`, `uboot/build-uboot.sh`, `nm liben683sb.a`, `sim/LICENSE`(Rambus).



Rambus RT-130 Root of Trust 블록도(옛 이름 VaultIP)

출처: Rambus, RT-130 제품 페이지(rambus.com/security/root-of-trust/rt-130/), 2023

EN683 SDK의 보안 모듈 이름은 EIP-130이고, 시뮬레이터 라이선스는 Rambus다. Rambus가 공개한 RT-130(옛 VaultIP) 블록도가 SDK 구성과 그대로 맞는다. 다만 EN683에 들어간 IP가 정확히 이 RT-130 버전인지는 벤더 문서로 확인하지 못했다.

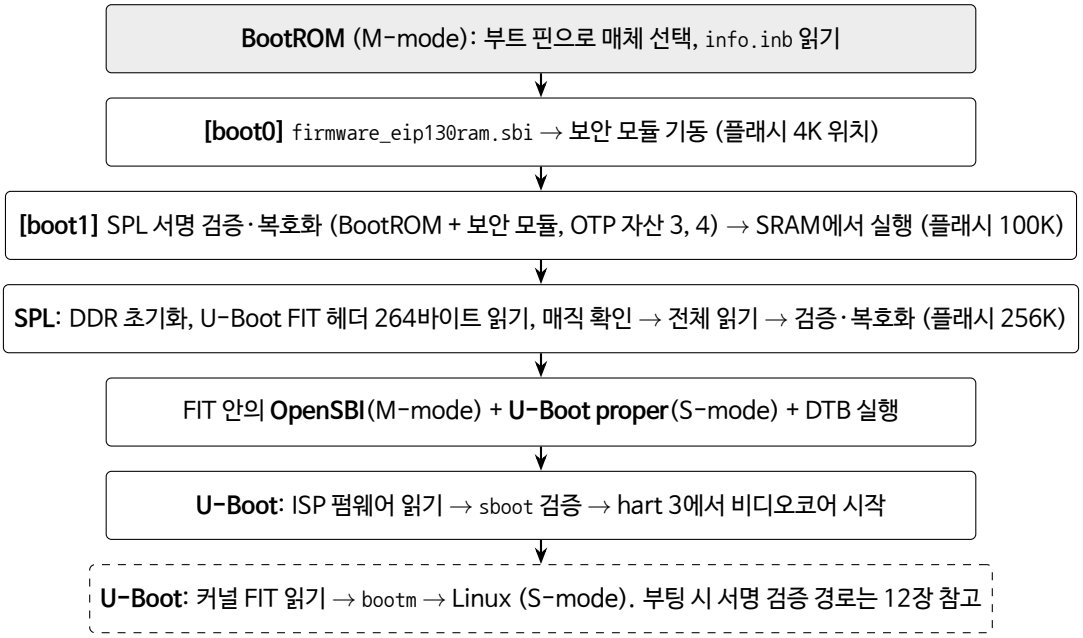
블록도	EN683 SDK에서 대응하는 것
Sign Tool	`util/secure_boot/signtool`
Secure Boot library, ROM의 SB Library	`conf/liben683sb.a`(SPL, U-Boot에 링크), BootROM
Secure Mailbox	CPU가 보안 모듈에 토큰을 보내는 통로. `Eip130Token_*`, U-Boot의 MBOX 레지스터
μC, Program ROM/RAM	보안 모듈 자체 CPU. RAM에 올리는 펌웨어가 `firmware_eip130ram.sbi`
Secure AssetStore	키를 참조 번호로만 쓰고 값은 내주지 않는 저장소. 검증 함수의 `pkidx=4`
Crypto Dataplane(AES, SHA-2, ECC)	BLw 복호화(AES-256), SHA-256 해시, ECDSA P-256 검증
OTP Interface, OTP module	RoT-OTP. Device Unique Key = HUK, Boot Keys = SBPKD/SBCR
Mon. Counters	단조 증가 카운터. 롤백 방지에 쓰는 기능(6장). EN683 기본 설정에서 사용 여부는 확인하지 못했다

이 그림에서 중점적으로 봐야 할 것은 **CPU와 보안 모듈 사이에 Secure Mailbox만 있다**는 점이다. CPU는 키 값을 직접 읽을 수 없고, 메일박스로 "이 이미지를 4번 키로 검증하라"는 요청만 보낸다. 그래서 리눅스나 U-Boot가 해킹되어도 OTP의 비밀 키는 밖으로 나오지 않는다. 오른쪽 아래의 화살표는 ROM의 SB Library가 OTP의 Boot Keys를 기준으로 쓴다는 뜻이다.

중요한 점은 **서명 검증과 복호화를 CPU 코드가 아니라 보안 모듈이 한다**는 것이다. 라이브러리 심볼에 `Eip130Token_Command_WriteByteArray`가 있는 것으로 보아, SPL과 U-Boot는 검증 요청을 토큰 형태로 보안 모듈에 보내고 결과를 받는다. 복호화 키(SBCR)는 RoT-OTP 안에 있고 보안 모듈 밖으로 나오지 않는다. 그래서 CPU에서 도는 코드가 해킹되어도 복호화 키를 읽어 갈 수 없다.

이 구조 때문에 시큐어 부트 모델에서는 BootROM이 [boot0]으로 **보안 모듈 펌웨어를 가장 먼저** 올린다. 보안 모듈이 살아 있어야 SPL을 검증할 수 있기 때문이다.

8. Boot Flow on EN683 — 전원부터 커널까지



출처: `script/create_ini.sh`, `script/sb_make_uboot.sh`, `uboot/common/spl/spl_spi.c`, `uboot/board/eyenix/common/en67x_common.c`. 오프셋은 NOR 시큐어 부트 빌드 기준.

8.1 Flash Layout — 무엇이 어디에 있나

플래시 오프셋	내용	비고
0x000000	`info.inb`	BootROM 부트 설정. 매직 0xec608d3f, 워치독 시간, 로그 UART, [bootN] 항목
0x001000 (4K)	`firmware_eip130ram.sbi`	보안 모듈 펌웨어, 96448바이트
0x019000 (100K)	서명된 SPL	`u-boot-spl-sign.bin`. BootROM의 lbase 0xC0019000 = QSPI 매핑 주소 + 100K
0x040000 (256K)	서명된 U-Boot FIT	`u-boot-sign.bin`(OpenSBI + U-Boot + DTB)
0x0C0000	커널 FIT (10MB)	`uImage.en683.itb`. 시큐어 빌드에서는 서명된 파일로 바뀐다
0xAC0000	ISP 펌웨어 (1344KB)	비디오코어 펌웨어. 시큐어 빌드에서 서명
0xC10000	MAC 정보 (64KB)	서명 없음
0xC20000	App (19.8MB)	**서명 없음**

출처: `sb_make_uboot.sh`(INPUT_FILE0~3_POS), `en683-ref.ext.h`, `board_X.cfg`. 일반 빌드에서는 0x0에 SPL과 U-Boot가 다른 방식으로 묶인다.

> 오프셋 네 개가 여러 파일에 흩어져 있다는 점을 중점적으로 봐야 한다. `sb_make_uboot.sh`의 100K, 256K는 U-Boot 설정 `en683-ref.h`의 `TFTP_UBOOT_SPL_BASE`(0x85019000), `TFTP_UBOOT_BASE`(0x85040000)와 defconfig의 `CONFIG_SYS_SPI_U_BOOT_OFFS`와 맞아야 한다. SDK 주석도 하나를 바꾸면 나머지를 함께 바꾸라고 적고 있다. SPL이 커져서 100K~256K 범위를 넘으면 `check_file_size`에서 빌드가 실패하는데, 이것을 피하려고 오프셋을 한 곳만 바꾸면 SPL이 엉뚱한 위치에서 U-Boot를 읽어 `Fail case 1`(매직 불일치)로 멈춘다.

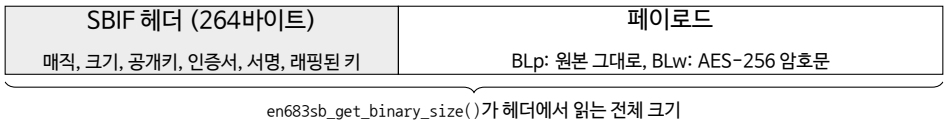
8.2 EVK Boot Log — 지금 보드에서 보이는 것

```
LPDDR4-2664Mbps 32-bits 4Gb*2 (1024MByte)      <- SPL: DDR 초기화
Trying to boot from SPI                         <- SPL: U-Boot를 SPI에서 읽음
U-Boot 2022.10 (Oct 21 2025 - 17:17:56 +0900)   <- 시큐어 빌드라면 이 앞에 "Secure-Boot Verify OK"
...
en683 ISP boot start                            <- ISP 펌웨어 (시큐어 빌드: sboot 검증)
Reading 1376256 byte(s) at offset 0x00ac0000
Reading 10485760 byte(s) at offset 0x000c0000   <- 커널 FIT 읽기
## Loading kernel from FIT Image at 85000000 ...
   Verifying Hash Integrity ... crc32+ md5+ OK  <- 무결성 확인일 뿐, 서명 검증 아님
Starting kernel ...
[    0.000000] SBI specification v2.0 detected <- OpenSBI(M-mode) 위에서 커널(S-mode)
```

출처: `~/en683_bsp/work/en683-boot.log`. 이 보드는 `Cross.param`의 `USE_SECUREBOOT="N"`으로 빌드되어 있다.

> 지금 로그에는 `Secure-Boot` 문구가 하나도 없다. SB 옵션을 켜고 빌드하면 SPL이 U-Boot를 검증한 뒤 `Secure-Boot Verify OK`를 출력하고(`spl_spi.c`), ISP 펌웨어를 올릴 때도 `sboot`가 같은 문구를 출력한다. 로그로 시큐어 부트가 실제로 동작하는지 확인하려면 **이 문구가 단계마다 나오는지**를 봐야 한다. 시큐어 빌드 로그를 확보하면 이 페이지의 주석을 실제 값으로 바꿔야 한다.
>
> OpenSBI는 U-Boot FIT 안에 함께 들어 있다. 그래서 SPL이 U-Boot FIT 전체를 검증하면 M-mode에서 도는 OpenSBI도 함께 검증된다. RISC-V에서 M-mode는 가장 높은 권한이므로, OpenSBI가 체인 밖에 있으면 커널을 검증해도 소용이 없다.

9. Image Format & Verification — 서명된 이미지 안에는 무엇이 있나



헤더 크기는 `spl_spi.c`의 주석(sbif header is 264 bytes)을 따랐다. 헤더 안 필드의 순서와 크기는 공개되지 않았다.

매직	값(ASCII)	의미
SB_MAGIC_BLP	0x424c7002 (“BLp” + 02)	서명만. 페이로드는 평문
SB_MAGIC_BLW	0x424c7702 (“BLw” + 02)	서명 + 암호화. AES 키를 래핑해 헤더에 넣고, SBCR로 언랩
SB_MAGIC_BLE	0x424c6502 (“BLe” + 02)	검증 코드가 허용하는 형식. SDK 빌드 스크립트는 만들지 않는다
SB_MAGIC_BLX	0x424c7802 (“BLx” + 02)	같은 상황. 시뮬레이터 도움말에 BLx 키 옵션이 따로 있다

출처: `uboot/Makefile` 709~712행, `board_X.cfg` 주석(blp signature only, blw signature+encryption), `sim/securebootsim_*` 도움말. BLe, BLx의 정확한 용도는 확인하지 못했다.

9.1 Verification Steps — 검증 함수가 하는 일

1. **매직 확인.** 첫 4바이트가 위 네 값 중 하나가 아니면 서명된 이미지가 아니다. `Fail case 1`.
2. **크기 읽기.** 헤더에서 전체 크기를 읽고, 그만큼 플래시에서 다시 읽는다.

3. **CM 공개키 확인.** 빌드 때 `-pubkeytype image`로 CM 공개키를 헤더에 넣었다. 검증 함수의 첫 인자 `pkidx=4`는 **OTP 자산 4(SBPKD)**를 가리키는 것으로 보이며, 헤더 공개키의 해시를 이 값과 비교한다.
4. **DM 인증서 확인.** CM 공개키로 DM.cert.bin의 서명을 확인해 DM 공개키를 믿는다.
5. **이미지 서명 확인.** DM 공개키로 헤더와 페이로드의 ECDSA P-256 서명을 확인한다.
6. **복호화(BLw).** 헤더의 래핑된 AES-256 키를 보안 모듈이 SBCR(자산 3)로 풀고, 페이로드를 복호화한다(라이브러리의 `AES_IF_CBC_Decrypt`, `AES_IF_Ctx_LoadWrappedKey`).
7. **속성 확인.** `SBIF_Attribute_Check`. 실패하면 `Fail case 2`.
8. **형식 확인.** SPL에서는 결과가 FIT(매직 0xd00dfeed)인지 본다. 아니면 `Fail case 3`.

3~7의 순서는 `sb_bingen.sh`가 `signtool`에 주는 인자, 라이브러리 심볼 이름, 시뮬레이터 문자열(Public key in OTP/ROM/image header, unwrap key)에서 추론했다. 벤더 문서로 확인해야 한다.

> SDK에는 **PC에서 검증을 미리 해 볼 수 있는 시뮬레이터**(`util/secure_boot/sim/securebootsim_*`)가 있다. 도움말에 따르면 공개키 파일과 서명된 이미지를 넣으면 헤더를 출력하고 검증하며, BLw 이미지는 언랩 키 파일을 함께 주면 복호화 결과까지 쓴다. 보드에 올리기 전에 이 도구로 서명 결과를 확인하면, 보드에서 `Fail case 2`가 났을 때 원인이 이미지인지 보드의 OTP인지 나눌 수 있다. 시뮬레이터는 Rambus 라이선스가 걸린 바이너리이므로 외부로 배포하면 안 된다.

10. Build & Sign — SDK에서 서명이 들어가는 곳

10.1 Switches

```
# Cross.param: 빌드 인자에 SB를 주면 켜진다
USE_SECUREBOOT="N"        # SB  -> "Y"
USE_ROTOTP_W="N"          # PROV -> "Y" (OTP provisioning 빌드)

# conf/en683X/board_X.cfg
SB_MAKER="DM"                          # 로컬 서명: keyset/DM.der 사용
# SB_MAKER="REMOTE:192.168.0.98:50683" # 원격 서명 서버 사용
SB_MODE_UBOOT_SPL="blw"   # blp(서명만) / blw(서명+암호화)
SB_MODE_UBOOT="blw"
SB_MODE_KERNEL="blw"
SB_MODE_FIRMWARE="blw"
```

10.2 What the Makefile Signs

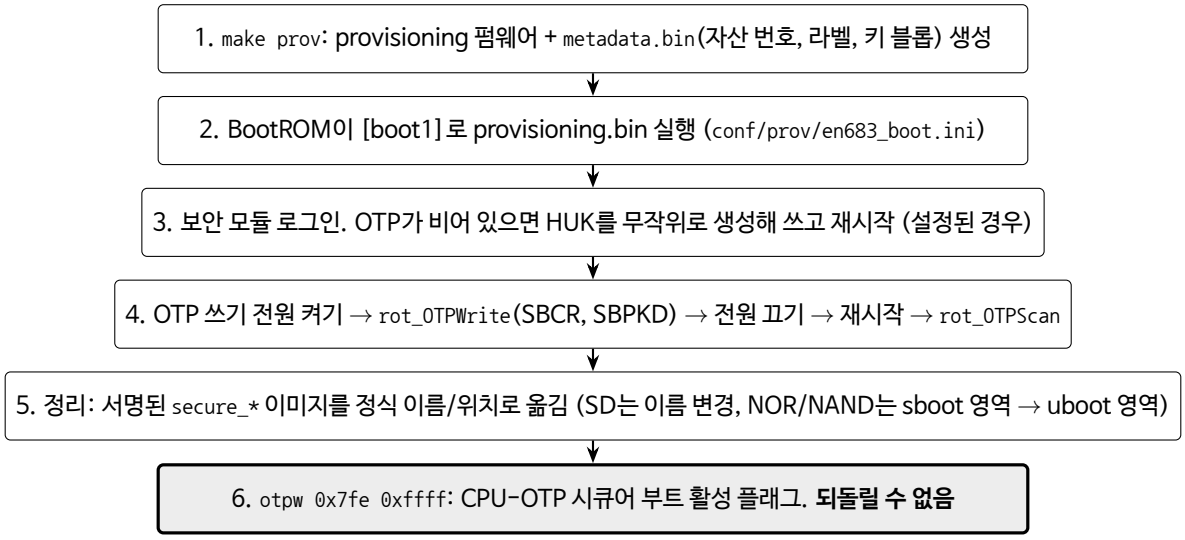
대상	Makefile 타깃	결과 파일
SPL	`sbuboot: sb_bingen.sh DM blw u-boot-spl.bin`	`u-boot-spl-sign.bin`
U-Boot FIT	`sbuboot: sb_bingen.sh DM blw u-boot.itb`	`u-boot-sign.bin`. 이어서 `sb_make_uboot.sh`가 info.inb, EIP-130 펌웨어와 함께 0/4K/100K/256K로 묶음
커널 FIT	`uImage: mkimage` 후 `sb_bingen.sh`	`uImage.en683-sign.itb` → `build/uImage.en683.itb`
ISP 펌웨어	`firmware: sb_bingen.sh`	`<firmware>-sign.<ext>`
ramdisk, App	서명 단계 없음	그대로

```
# script/sb_bingen.sh, BLw 로컬 서명 (요약)
signtool sign -type BLw -pubkeytype image \
    -infile  <입력> \
    -cert    keyset/DM.cert.bin      # CM이 서명한 DM 인증서
    -algo    aes256 \
    -pubkey  keyset/SBEN683.pub.bin  # CM 공개키 (헤더에 들어감)
    -prikey  keyset/DM.der           # DM 개인키로 서명
    -wrapkey keyset/SBEN683.puk.bin  # = SBCR, AES 키를 감싸는 키
    -outfile <출력>
```

인자 하나하나가 앞 장의 개념과 이어진다. `-pubkey`는 OTP 해시와 비교될 CM 공개키, `-cert`는 CM이 DM을 보증한 인증서, `-prikey`는 실제 서명에 쓰는 DM 개인키, `-wrapkey`는 OTP 자산 3(SBCR)과 같은 값이다. 즉 **SBCR은 빌드하는 PC와 칩의 OTP 양쪽에 같은 값이 있어야** 복호화가 된다. 이 값이 빌드 서버에 평문 파일로 있으면 그 파일이 유출될 때 암호화의 의미가 없어진다.

원격 서명(`SB_MAKER="REMOTE:IP:포트"`)을 쓰면 개발 PC에는 개인키가 없고, 서명 서버만 키를 갖는다. 키를 한 곳에 모으는 좋은 방법이다. 다만 SDK의 예제 서버(`sign_server/server.py`)는 Flask로 **0.0.0.0의 50683 포트에서 인증 없이** 요청을 받는다. 같은 네트워크의 누구나 임의 바이너리를 정식 서명받을 수 있다는 뜻이므로, 그대로 쓰지 말고 접근 통제와 요청 기록을 추가해야 한다.

11. Provisioning — OTP에 키를 쓰고 시큐어 부트를 켜는 순서



출처: Makefile의 prov 타깃, conf/prov/make_metadata.sh, uboot/board/eyenix/en683-ref/en683_prov.c.

순서에서 가장 중요하게 봐야 할 것은 **활성 플래그를 맨 마지막에 쓴다**는 점이다. 키를 OTP에 쓰고, 서명된 이미지를 부트 위치에 둔 다음에야 플래그를 켠다. 순서가 바뀌면 다음 부팅에서 BootROM이 서명 안 된 SPL을 만나거나 OTP 해시와 맞지 않는 이미지를 만나 멈추고, OTP는 되돌릴 수 없으므로 보드를 살릴 수 없다.

OTP 쓰기에는 **별도의 쓰기 전원**이 필요하다. 코드에서 OTP_IOCTL_PWEN, PWDIS로 켜고 끄고, 디바이스트리 주석도 같은 내용을 적고 있다. 쓰기 중에 전원이 흔들리면 일부 비트만 쓰일 수 있으므로, 양산 지그에서는 이 전원의 안정성과 쓰기 후 rot_OTPScan 결과 확인을 공정 항목으로 넣어야 한다. 전원 레일 측정에 관한 내용은 "Power & Memory Troubleshooting" 문서를 함께 보면 된다.

EVK에서는 이 과정을 실행하지 않는다. 프로젝트 문서(FLASHING.md)도 U-Boot에서 otp_write 같은 OTP 명령과 시큐어 부트 명령을 실행하지 말라고 적고 있다. 시큐어 부트 흐름은 SB 옵션 빌드, 시뮬레이터, 그리고 아래의 sboot 검증 명령까지로 공부하는 것이 안전하다.

12. Gaps & Checks — 체인이 끊어지는 곳

SDK 소스를 따라가면서 확인한 것과 확인하지 못한 것을 나눠 적는다. 실제 제품에 적용하기 전에 아래 항목을 하나씩 확인해야 한다.

항목	소스에서 본 것	확인할 것
커널 부팅 시 검증	빌드 때 커널을 BLw로 서명한다. TFTP 업데이트(tbk)에서는 sboot로 검증한다. 그런데 부팅 경로 do_kernel_start()는 커널을 읽은 뒤 바로 bootm을 부른다	암호화된 커널은 bootm이 읽을 수 없으므로, 시큐어 빌드에서 이 경로가 실제로 어떻게 동작하는지 SB 빌드 부팅 로그로 확인해야 한다
App, ramdisk	서명 단계가 없다. tba도 검증 없이 플래시에 쓴다	체인이 커널에서 끝난다. dm-verity, 서명된 squashfs, 업데이트 패키지 서명 같은 다음 단계가 필요하다
데모 키	keyset의 개인키가 공개되어 있다	양산 전에 반드시 새로 생성. 기존 파일이 빌드 서버나 저장소에 남아 있지 않은지
서명 서버	인증 없이 0.0.0.0에서 수신	네트워크 분리, 인증, 요청 기록
롤백 방지	속성 검사 함수는 있으나 버전 비교 설정은 찾지 못함. SFWBVER는 주석 처리	벤더 문서로 버전 필드와 OTP 카운터 사용법 확인
디버그	Secure JTAG 키가 Eyenix 기본값. U-Boot 콘솔 열림	자기 키로 교체, 콘솔 인증 또는 비활성화

이 표는 **소스 코드만 읽고 정리한 것**이다. 라이브러리가 바이너리이고 벤더 보안 문서를 보지 못했으므로, 첫 행처럼 의문이 남는 부분은 단정하지 않았다. 특히 커널 검증은 시큐어 부트에서 가장 중요한 단계이므로, SB 빌드를 실제로 부팅해 `Secure-Boot Verify OK`가 커널 앞에서도 나오는지 반드시 확인해야 한다.

13. Troubleshooting — 실패 메시지 읽기

시큐어 부트가 실패하면 그 단계에서 부팅이 멈추고 시리얼에 메시지 하나만 남는다. 메시지는 SPL과 U-Boot의 검증 코드(`spl_spi.c`, `cpu.c`)가 출력하며, 번호마다 실패한 지점이 다르다. 먼저 **어느 단계에서 멈췄는지**, 다음으로 **몇 번 case인지**를 봐야 한다.

13.1 Fail Messages — 번호별 원인

메시지	뜻	먼저 볼 것
`Fail case 1`	첫 4바이트가 BLp/BLw/BLe/BLx 매직이 아니다. 서명 안 된 이미지이거나 엉뚱한 위치를 읽었다	서명 안 된 파일을 썼는지. 100K/256K 오프셋, defconfig, TFTP 주소가 서로 맞는지 (8.1절)
`Fail case 2`	서명 검증이나 복호화가 실패했다	다른 키로 서명했는지(SB_MAKER, keyset). OTP의 SBPKD/SBCR과 빌드 키가 같은 세트인지. 이미지가 잘렸는지
`Fail case 3`	복호화는 됐지만 결과가 FIT 형식이 아니다 (SPL 단계)	SPL 자리에 U-Boot를 넣었거나 그 반대인지. `u-boot.itb` 대신 `u-boot.bin`을 서명했는지
출력 없음	BootROM 단계에서 멈췄다. SPL 검증 실패나 보안 모듈 펌웨어 문제	`info.ini`의 [log] level을 올려 BootROM 로그 확인. [boot0]의 크기와 위치

13.2 sboot — 보드에서 검증만 해 보기

시큐어 빌드의 U-Boot(`CONFIG_EN683_SECURE_BOOT`)에는 메모리에 올린 이미지를 검증만 해 보는 `sboot` 명령이 들어간다. 인자는 두 개다. 첫째는 서명된 이미지가 있는 주소, 둘째는 검증과 복호화 결과를 쓸 주소다.

```
=> tftp 0x85000000 uImage.en683.itb
=> sboot 0x85000000 0x86000000
Secure-Boot Verify OK
```

출처: `uboot/arch/riscv/cpu/en683/cpu.c`의 `do_sboot()`. 이 명령은 메모리에서 검증만 하고 OTP를 쓰지 않는다. 그래도 EVK에서는 프로젝트 규칙(`FLASHING.md`)을 먼저 따른다.

> 현장에서 `Fail case 2`를 만나면 **키 세트가 섞였는지**를 가장 먼저 의심해야 한다. 데모 키로 서명한 이미지를 양산 키가 OTP에 들어간 보드에 넣거나, 개발자마다 keyset 폴더가 달라서 생기는 경우가 많다.
> 원인을 빨리 나누려면 빌드 산출물마다 **서명에 쓴 CM 공개키 해시와 DM 공개키 해시를 기록**해 두어야 한다. 보드에서 읽은 OTP 값과 비교하면 이미지 문제인지 보드 문제인지 바로 구분된다. 서명 서버의 `calculate_pubkey_hash()`가 같은 목적의 함수다. 보드에 올리기 전에 PC의 시뮬레이터(9장)로 같은 이미지를 검증해 보는 것도 같은 효과가 있다.

14. Glossary & References

용어	뜻
RoT	Root of Trust. 검증 없이 믿는 출발점(BootROM + OTP + 보안 모듈)
CRTM	Core Root of Trust for Measurement. 첫 번째로 실행되는 바꿀 수 없는 코드
OTP	한 번만 쓸 수 있는 메모리. 퓨즈
HUK	Hardware Unique Key. 칩마다 다른 비밀 키
KEK / 키 래핑	키를 다른 키로 암호화해 보관하는 것. SBCR이 KEK 역할
SBIF	EN683 서명 이미지 형식(Secure Boot Image Format으로 보인다)
BLp / BLw	서명만 / 서명+암호화 이미지
CM / DM	Chip Manufacturer / Device Manufacturer 키
FIT	U-Boot의 이미지 묶음 형식(커널, DTB, 해시를 한 파일에)
M-mode / S-mode	RISC-V 권한 모드. OpenSBI는 M, U-Boot proper와 커널은 S

- EN683 SDK: `Makefile`, `Config.mk`, `Cross.param`, `conf/en683X/board_X.cfg`, `script/sb_bingen.sh`, `script/sb_make_uboot.sh`, `script/create_ini.sh`, `util/secure_boot/`(README, WARNING.txt, keygen, sign_server, sim), `conf/prov/`, `util/gen_rotfw/config/customization.cfg`.
- EN683 U-Boot 2022.10: `common/spl/spl_spi.c`, `spl_fat.c`, `spl_ymodem.c`, `arch/riscv/cpu/en683/cpu.c`, `board/eyenix/common/en67x_common.c`, `board/eyenix/en683-ref/en683_prov.c`, `include/configs/en683-ref.h`.
- EVK 부트 로그: `~/en683_bsp/work/en683-boot.log`. 프로젝트 문서: `~/wisecon/en683-evk-secure-ipcam/docs/`.
- EN683 U-Boot 소스의 `doc/uImage.FIT/verified-boot.txt`: FIT 서명 방식의 verified boot(EN683 방식과 비교용).
- C. Kothandaraman, S. K. Iyer, S. S. Iyer, “Electrically Programmable Fuse (eFUSE) Using Electromigration in Silicides,” IEEE Electron Device Letters, vol. 23, no. 9, pp. 523 - 525, 2002.
- S. N. Chakravarti et al., “MOSFET Fuse Programmed by Electromigration,” US Patent 7,531,886 B2, IBM, 2009.

- Rambus, RT-130 Root of Trust 제품 페이지와 블록도, rambus.com/security/root-of-trust/rt-130/.
- NIST SP 800-193, “Platform Firmware Resiliency Guidelines,” 2018 (Figure 2: Roots of Trust).
- NIST FIPS 186-5(ECDSA), FIPS 180-4(SHA-256).
- RISC-V Privileged Architecture Specification(M/S 모드), OpenSBI 문서.
- 함께 볼 문서: “Power & Memory Troubleshooting”(부트 체인과 전원 순서), “EN683 NPU 실시간 탐지”.